

ASSIGNMENT 2B

CL1-07-Ru-Group04

GitHub Link: <https://github.com/ruvxn/Traffic-Guidance-System.git>

Student Name	Student ID	Contributions
Ananda Pathiranage Ruveen Thathsilu Jayasinghe	104317649	• LSTM model • Preprocessing scripts • Map edge manual extraction • Map coordinates extraction and fixes • Map visualisation with folium • Distance calculation script • Travel time calculation script • Traffic volume prediction script • Yen's algorithm • main.py • app.py
Denver J Cope	104738758	• GRU model • ASTAR algorithm implementation
Fayiz Kallupalathingal	104658733	• RNN model • Index.html • Result.html
Rahat Alam	103810105	• TCN model • Styles.css

Table of Contents

Introduction.....	3
Instructions on how to use program.....	3
Models	3
Features/Bugs/Missing	4
Testing.....	5
Insights.....	7
Research.....	8
Conclusion	9
Acknowledgment and Resources.....	10
References.....	10

Introduction

This part of the assignment is the advanced version of the tree-based search system. The aim of this project is to build a system that can help people choose the fastest route in the city of Boroondara on the basis of predicted traffic. Machine learning (ML) models are used to analyse the given data, forecast the traffic and determining which route is the fastest and the time of travel based on the origin and destination and the time of travel. The data used for this assignment is provided by Vic roads which contains traffic flows information (how many cars pass through intersections every 15 minutes).

Machine learning models such as LSTM (Long Short-Term Memory), RNN (Recurrent Neural Network), GRU (Gate Recurrent Unit), TCN (Temporal Convolutional Networks) are used to achieve the goal of this assignment. Each machine learning model is compared and selected LSTM to do the prediction as it has all-round good accuracy.

The final system is combined the ML predictions with the graph-based search system. A simple GUI is also included to allow users to input origin and destination points, see the fastest route in the map, estimated time of travel. Even though this system is not as advanced as google maps, this demonstrates how artificial intelligence can be used to solve traffic routing problems in a practical way.

Instructions on how to use program

1. Clone the Repository

```
git clone https://github.com/ruvxn/Traffic-Guidance-System.git
```

```
cd Traffic-Guidance-System
```

2. Create a Virtual Environment

```
python3 -m venv TBRGSvenv
```

```
source TBRGSvenv/bin/activate # On Windows: TBRGSvenv\Scripts\activate
```

3. Install Required Packages

```
pip install -r requirements.txt
```

4. Run the Web Application

```
python app.py
```

Then open your browser and go to:

<http://127.0.0.1:5000/>

Models

LSTM

Long Short-Term Memory (LSTM) is a form of a Recurrent Neural Network (RNN) that mitigates the vanishing gradient problem (Zhang et al., 2020). Vanishing gradient is a problem that emerges in RNNs during the process of backpropagation which affects the training efficiency of the model (GeeksforGeeks, 2025). LSTM handles this error by introducing memory cells and gates for input, forget and output functions that effectively manages the consistent flow of information. Due to the established gates controlling consistent holding of valuable information LSTM models are well suited for time series prediction scenarios.

GRU

Gate recurrent unit (GRU) is another type of RNN, which serves the purpose of handling the vanishing gradient problem. It closely resembles the LSTM model however simplifies the architecture by only using 2 gates (Update and Reset). Whilst often comparable to the LSTM in performance, the fewer gates result in along with merging the hidden and cell states, make GRU models faster to train.

The reset gate determines how much of the previous hidden state contributes to the new candidate state, unlike basic RNNs, which overwrite information completely, this gate dynamically preserves and discards past data. It combines the current input with the prior hidden state, then applies weights and bias's before squeezes the result into a range between 0 and 1 by using a sigmoid function. This range determines percentage of data to ignore/keep. (GeeksForGeeks, 2019)

The update gate determines the percentage of data that should be passed to the hidden state. To do so it uses a similar formula as the reset gate, however using different bias/weight values.

The candidate hidden state is a new potential memory that would be used for updating the current hidden state and is only used when combined with the update gate, this value will always be between -1 and 1 due to its usage of the tanh function.

The final hidden blends the old hidden state and the candidate. This dynamic balancing allows GRU's to retain long-term dependencies without the issues of vanishing gradients. (Medium, 2025)

RNN

Recurrent Neutral Network (RNN) is a deep learning model which is trained to process and convert a sequential data into sequential output. The speciality of RNN is that it can remember information from earlier steps and use it in next predictions. They struggle to learn long sequence patterns which makes it hard for the model to update its memory during training, this is called vanishing gradient. So even though RNNs are not preferable to use in tasks that need long-term memory, it is useful in that only need short-term memory.

TCN

TCN works by using convolutions, instead of recycling the same computation by layers, to process sequential data. TCNs unlike RNNs apply causal and dilated convolutions which means they can identify longer connections more efficiently without the vanishing gradient issue. Memories formed within TCNs from earlier events are accessed through wide receptive fields, allowing them to succeed at tasks requiring several timesteps. Their convolutional structure makes it easy to both train them and use them in parallel. From the evaluation, TCN performed much better than RNN, LSTM and GRU in accuracy and speed of training, so it's a strong choice for time-series forecasting.

Features/Bugs/Missing

Features

1. Traffic Volume Prediction:

We have trained 4 different models, LSTM, GRU, RNN and TCN on historical time series traffic flow data. However, LSTM is used for the predictions since performance accuracy was comparatively good and stable. This model predicts traffic volume for a SCATS site at a given time.

2. Travel Time Estimation:

This was achieved by solving a quadratic formula for traffic flow vs speed provided with assumptions by Intelligent Systems Team (2025). Solves the equation and takes maximum value assuming free flow of traffic. If traffic flow is less than or equal 351 vehicles per hour, speed is set to be a constant 60km/h. A minimum speed of 5km/h is set for extremely congested traffic scenarios. A constant of 30 seconds added to handle constant delays.

3. Distance Calculation:

The haversine formula was used to calculate distance between two SCATS sites. This choice was made due to the coordinates of SCATS sites being in longitudinal and latitudinal values and the haversine formula accurately handles these values by considering the earth's curvature (GeeksforGeeks, 2022).

4. Top 5 route calculation

The Yen's K-shortest paths algorithm was used to retrieve the top 5 fastest routes. A star algorithm from assignment 2A was implemented by restructuring the format of the function to integrate to the main application. By setting $k=5$ in the yen's algorithm we were able to retrieve the top 5 fastest routes.

5. Visualisation Map

Folium library was used for the implementation of the map. Further information regarding this is presented in the research segment.

Bugs

1. Coordinates of the SCATS sites were originally off by a constant value when compared to google maps. Later this was resolved in majority of cases by adding a constant calculated offset for longitude and latitude values. However, some SCAT sites might still be a slightly off centred to the actual position. Due to these offsets, the route visualisations might slightly overlap.
2. The prediction change on different times is not significant.
3. No connection shown between 2200 and 3126 SCATS.

Missing

1. Ability to choose between different prediction models is not integrated. Our application solely uses LSTM model for predictions.

Testing

Test case 1: Origin = 970, Destination = 3812

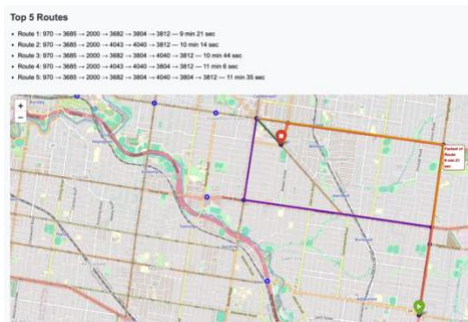


Figure 1: Test case scenario 1 visualisation

The shortest route seems to be accurate at 9 minutes and 21 seconds.

The algorithm has found different node patterns to reach the given goal in 5 different ways.

Test case 2: Origin =970, Destination = 2820

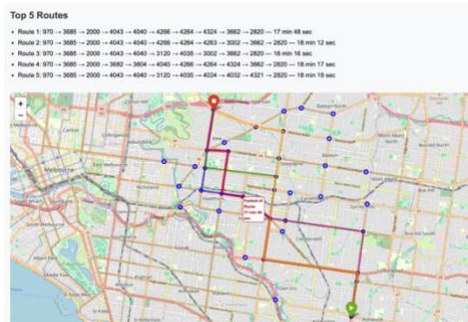


Figure 2: Test case scenario 2 visualisation

The shortest route seems to be accurate at 17 minutes and 21 seconds. This test was created to test the application on two SCATS furthest apart from each other. This shows the yen's algorithm in action because when we observe the map, it is evident that there are multiple other potentially longer ways to reach the destination. However, only the shortest/fastest paths have been selected.

Test case 3: Origin= 970, Destination =3685

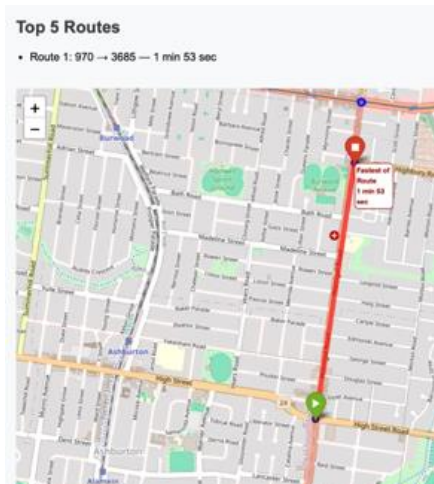


Figure 3: Test case scenario 3 visualisation

The only available path was taken.

This test was created to test a scenario where only 1 path was possible.

The application handles this successfully by only providing routes for only available path.

Test case 4: Origin= 970, Destination= 970

No path is shown in the visualisation since there is not route when destination and origin is same. The estimates time is 0 minutes and 0 seconds.

Test case 5: Origin = 2820, Destination =4264



Figure 4: Test case scenario 5 visualisation

This test was done to test the ability of finding efficient paths where multiple paths are available in a shirt area.

The shortest/fastest path found seems to be accurate.

Test case 6: Origin =970, Destination = 3002

This test case was created to test how the routes will be expanded differently in instances such as 3-way split roads at SCAT point 4043.

The routes don't expand from 4043 to 4273 in this case as that path would have a higher cost.

Test case 7: Origin = 4040, Destination = 4034

This test case was created to identify how the routes would expand when the path is a simple straight line. This uncovered how the routes will expand to the side but reconnect back to the main straight route.

Test case 8: Origin = 4812, Destination = 2846

This test case was done to test how two SCAT sites that were on the same edge that did not have a direct connection would show routes. We can see a very complex route expansion in this case, which finally successfully leads to the correct destination through the use of internal routes.

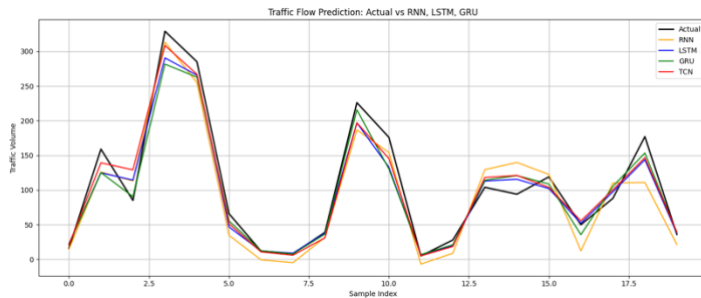
Test case 9: Origin = 2200, Destination = 3126

An error was noted when testing this test case, the server seems to unexpectedly drop the connection. The reason seems to be no paths between SCATS sites. This is a potential error in the manual handling of the edges list.

Test case 10: Origin= 4040, Destination = 4072

This test case tests the behaviour when the destination and origin are neighbouring SCATS nodes. The shortest path seems to always take the direct connected route and more routes through other segments of the map is taken successfully.

Insights



All models were trained on the same dataset with no changes to ensure consistency during evaluation. The model performance was evaluated based on 4 main metrics,

Figure 5: Traffic flow prediction comparison to actual.

Mean Absolute Error (MAE) – Measures average error in original units (number of vehicles in this case). The lower the MAE the better the performance.

1. Means Squared Error (MSE) – This was used during training and evaluation because penalty for errors is higher as they are squared. The lower the MSE the better the model performance.
2. Root Mean Squared Error (RMSE) – This is used to bring MSE value back to original units for clear understanding.
3. R^2 – This metric explain how much of the data is explained through the model. The higher the R^2 the better the model performance.

With the final accuracy scores we can identify that,

RNN Performance:	
MAE	: 20.1285
MSE	: 986.9079
RMSE	: 31.4151
R^2	: 0.8682
LSTM Performance:	
MAE	: 16.5915
MSE	: 766.6681
RMSE	: 27.6888
R^2	: 0.8976
GRU Performance:	
MAE	: 16.4751
MSE	: 771.1941
RMSE	: 27.7704
R^2	: 0.8970
TCN Performance:	
MAE	: 15.6035
MSE	: 710.2924
RMSE	: 26.6513
R^2	: 0.9052

1. TCN outperforms every other model with the least MAE at 15.6, MSE of 710.3, RMSE of 26.7 and R^2 value of 0.9.
2. LSTM and GRU performance are very similar to each other. However, LSTM slightly outperforms GRU model.
3. RNN shows the poorest performance with the highest MAE score out of all the models at 20.1, highest MSE score at 986.9, highest RMSE score at 31.4 and lowest R^2 at 0.87.

Figure 6: Model performance metrics

The TCN outperformed every other model which explains its ability to long range temporal dependencies within the data through dilated convolutions. Both the LSTM and GRU model seems to perform well with the sequence modelling however some effect of vanishing gradient in longer sequences might still be affecting the performance. RNN's poor performance can be considered reasonable due to its low memory capacity and not being able to identify and carry patterns in long sequences.

Research

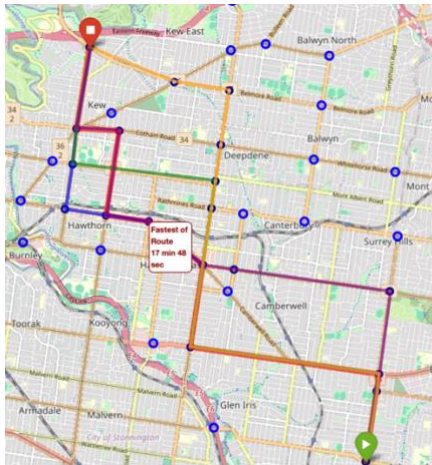


Figure 7: Map Visualisation

As a research section for assignment 2B, we implemented the SCATS map visualisation using Folium library in python. This was a challenging yet fun implementation that allowed us to visualise the SCATS and add interactivity to the map with hover and on-click functions.

After researching on other alternatives such as leaflet.js via JS and Plotly, we chose Folium for its simplicity and ease of python integration. Use of Folium allowed us to use OpenStreetMap map tiles without any issue. We implemented interactive markers for SCAT sites which on-click shows the SCATS number, coloured polylines for different routes that on-click shows the time estimation. During implementation we ran into

several problems,

1. At initial stage, the SCAT sites were plotted completely off to actual locations. This was fixed by comparing with google maps coordinates to adjust the offset. However, one SCATS site was originally showing entirely wrong coordinates, which was also manually adjusted separately by comparing with google maps coordinates.
2. When visualising multiple routes, it was originally challenging to distinguish one route from another even with different colours due to overlapping. This error was fixed by introducing a slight spacing between each route for better clarity when viewing each route.

We integrated a front end that allows the user to input the origin, destination and time and used Flask API endpoints to communicate these with the backend to then display the results.

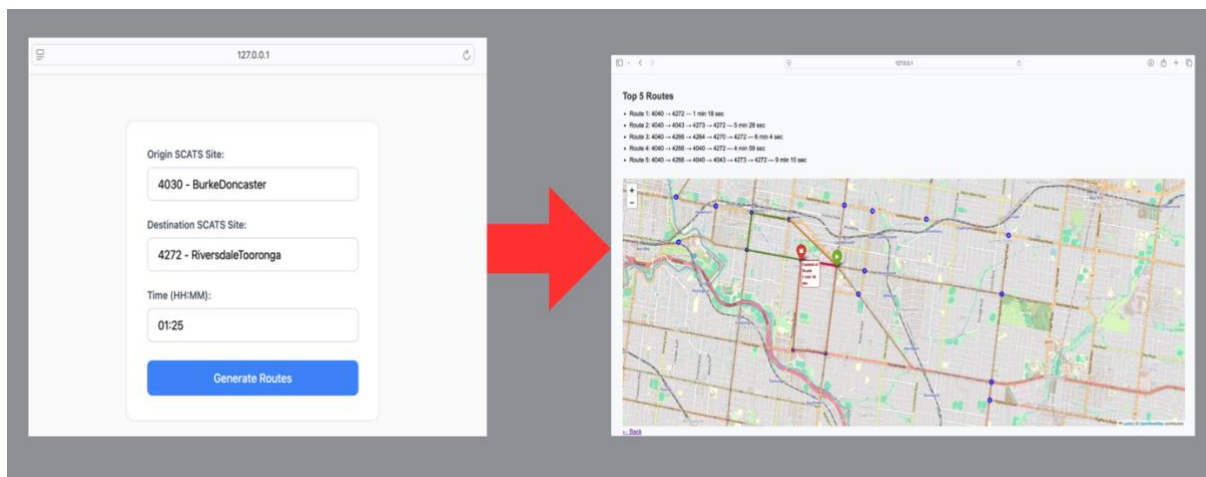


Figure 8: Visualization front end

Conclusion

The TBRGS project has underscored the transformative role of machine learning (ML) in modern technology, particularly in systems many take for granted such as smartphone navigation software. Whilst these tools often function as "black boxes" to end-users, this project has provided valuable insight into their inner workings.

Though the field of ML is far too vast to master in a single endeavour, the project highlighted the importance of selecting the right model for the task. Each approach carries unique trade-offs, and the choice hinges on the specific problem at hand. For our traffic prediction system, we required a recurrent neural network (RNN) capable of handling sequential data without succumbing to issues like the vanishing gradient problem. After evaluating alternatives, we opted for Long Short-Term Memory (LSTM) with Gated Recurrent Units (GRU) coming in a close second as we prioritized accuracy over training efficiency. Since our model didn't require frequent retraining, the longer training time of LSTM was an acceptable compromise.

Future improvements could focus on fine-tuning the training process and incorporating more recent data. For instance, relying on 2006 traffic data to model Melbourne's dynamic, ever-expanding road network in 2025 introduces inherent limitations. Updating the dataset and iterating on the model architecture would further enhance predictive performance, ensuring the system remains relevant in an evolving urban landscape.

Acknowledgment and Resources

LSTM: For understanding concepts and code implementation

https://d2l.ai/chapter_recurrent-modern/lstm.html

Intelligent Systems Team Time estimation script: To solve for time taken

<https://swinburne.instructure.com/courses/66745/files/37230209?wrap=1>

Haversine Function for distance calculation:

<https://www.geeksforgeeks.org/haversine-formula-to-find-distance-between-two-points-on-a-sphere/>

Folium interactive maps in python YouTube Instructions video:

https://youtu.be/2Mn6lvzUKvY?si=jVCwtXM5fF0hEQc_

Folium Markers: used to make SCATS sites

https://python-visualization.github.io/folium/latest/user_guide/ui_elements/popups.html

Folium Icons: used to mark origin and destination

https://python-visualization.github.io/folium/latest/user_guide/ui_elements/icons.html

Folium Polyline: Used to draw routes

https://python-visualization.github.io/folium/latest/user_guide/vector_layers/polyline.html

References

Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2020). Long short-term memory (LSTM). In Dive into deep learning. Retrieved May 23, 2025, from https://d2l.ai/chapter_recurrent-modern/lstm.html

GeeksforGeeks. (2025, April 3). Vanishing and exploding gradients problems in deep learning. <https://www.geeksforgeeks.org/vanishing-and-exploding-gradients-problems-in-deep-learning/>

GeeksforGeeks. (2022, September 5). Haversine formula to find distance between two points on a sphere. <https://www.geeksforgeeks.org/haversine-formula-to-find-distance-between-two-points-on-a-sphere/>

Singh, R. (2025, January 12). GRU Explained: The Simplified RNN Solution for Sequential Data. Medium. <https://ravjot03.medium.com/gru-explained-the-simplified-rnn-solution-for-sequential-data-c706d0d149c5>

Gated Recurrent Unit Networks. (2019, July 9). GeeksforGeeks. <https://www.geeksforgeeks.org/gated-recurrent-unit-networks/>